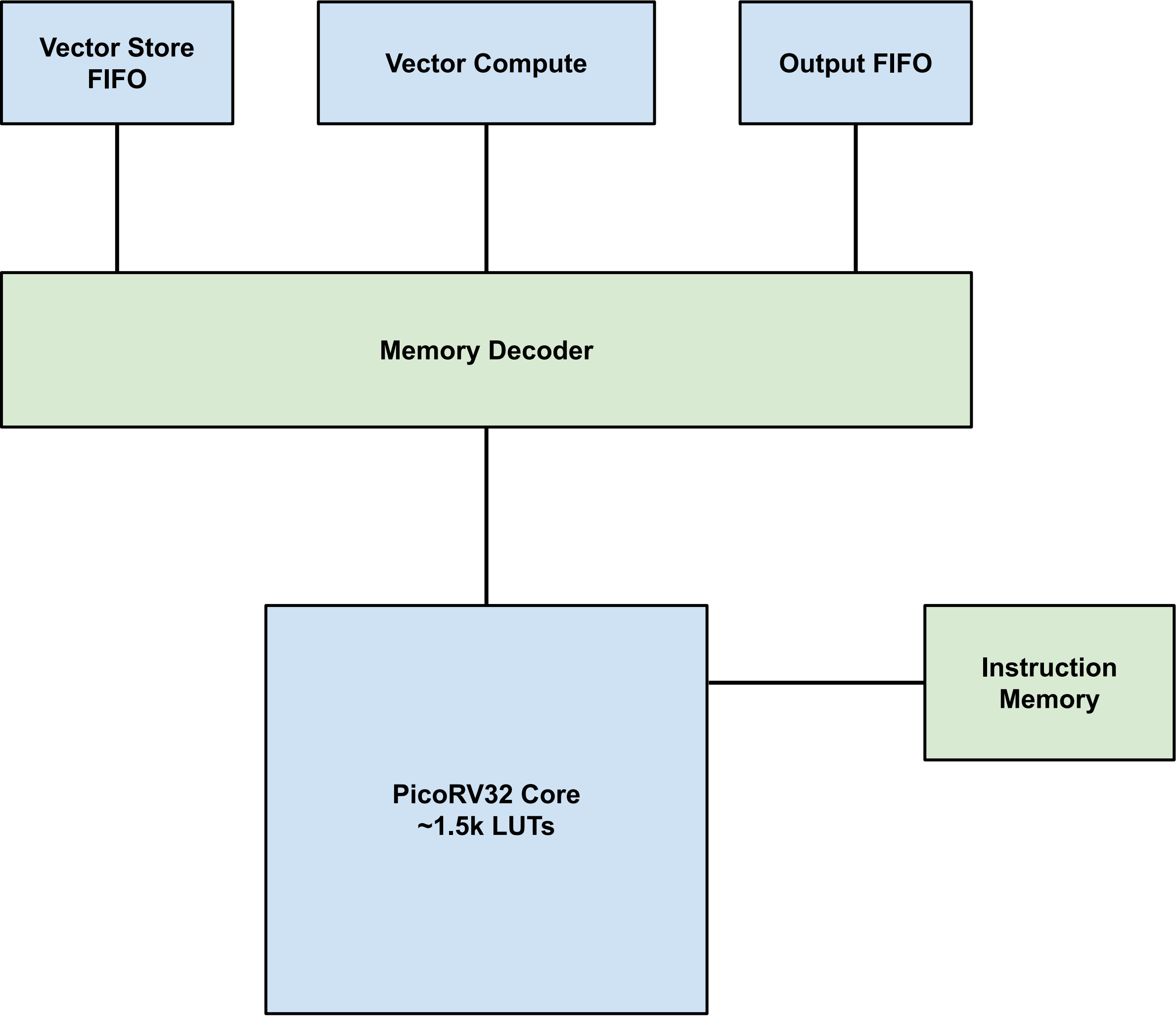


- What I need
- Small assembly file to init stack pointer and jump to main
 - Memory decoder
 - How is each vector going to be set
 - Memory Map
 - Interfaces between each block

- Quick FPGA Facts
- 176k LUTs
 - 640 DSP blocks
 - Multiples 19*18 natively, so 16*16 multiplication fits pretty nicely
 - 1280 BRAMs, 10kbit each
 - Efinix has their own Sapphire SoC but frick that
 -

- Computation Specs
- N=64, P=4 (number of MAC lanes, aka how many pairs can be computed at once). One lane, one DSP block
 - Packed 32 bit words
 - Word0 = {A[0], A[1]}
 - Word2 = {A[2], A[3]}
 - So essentially we are multiplying two 64 element vectors where each element is 16 bits. Each vector needs 32 words to fully store it. Since each word is 32 bits and we are packing so we can store two elements in one word
 - Result is 48 bits
 - Pop-on read FIFO, stores 16 values
 - Random Vector Generator
 - XORSHIFT in the RISC-V core to generate random vectors, seeded from a constant so it is reproducible

- Main.c Program Flow
- Generate random vector
 - Compute answer using C code
 - Generate answer using vector compute module -> store into fifo
 - Compare soft core answer with vector compute answer
 - Could print results to UART
 - Could blink an LED X amount of times if correct answer



- Improvements
- Tweak parameters of PicoRV32
 - Enable FAST_MUL
 - Word packed stores
 - XORSHIFT32 already generates a 32 bit vector, so just generate a 32 bit vector and use upper 16 bits and lower 16 as separate values, saves some cycles for load/store
 - Firmware re-order to start computation as soon as vector store FIFO is set
 - Look ahead ram
-
- Make this architecture its own core
 - Multiply massive amounts of data using these cores